Simulador de la línea de muebles

Qué hace cada archivo, dónde se cambia cada dato y cómo se corre · carpeta `03-simulacion/`

Regla de oro: ningún dato de la línea está escrito en el código Python

Las llegadas, los tiempos, las réplicas y la semilla viven en los archivos `escenarios/*.json`. En la demo se cambian en vivo desde el panel (`python panel.py`), sin tocar ni un archivo.

Qué hace cada archivo (patrón MVC)

ARCHIVO	QUÉ HACE	CAPA
main.py	Punto de entrada: lee `--config` y `--salida` y llama al controlador	Entrada
panel.py · animacion.py	Atajos de 5 líneas que abren la ventana del panel o la animación	Entrada
controlador/controlador.py	Valida todos los JSON, corre la simulación y guarda CSV y gráficos	Controlador
modelo/configuracion.py	Carga y valida un escenario: rechaza datos faltantes, negativos o vacíos	Modelo
modelo/simulacion.py	La línea en SimPy: llegadas, estaciones, réplicas, errores, Little y cuello	Modelo
modelo/teoria.py	Fórmulas M/M/1 (ρ, Lq, Wq) y capacidad de la línea	Modelo
vista/panel.py	Pantalla única: formulario, taller animado y resultados	Vista
vista/animacion.py	Dibuja el taller: operarios, colas y la estación cuello en rojo	Vista
vista/tablas.py · graficas.py	Tablas en la terminal y gráficos PNG	Vista
escenarios/*.json	`base.json` (E1), `e2.json`, `e3.json`; el panel guarda en `personalizado.json`	Datos
tests/ · salidas/	Pruebas con pytest · resultados generados (CSV y PNG)	Pruebas / salida

En el JSON o en el panel

DATO	EN EL JSON (ESCENARIOS/BASE.JSON)	EN EL PANEL
Llegadas por hora (λ)	`"tasa_llegadas_h": 3` · línea 3	Casilla “Llegadas por hora (λ)”
Tiempo de cada estación	`"tiempo_min"` de cada estación · líneas 5–8 (8, 10, 15, 12)	Columna “t (min)” junto a cada estación
Operarios por estación	`"recursos": 1` · líneas 5–8. Con más de 1 la teoría M/M/1 sale “—”	No se cambia: el panel fija 1 (M/M/1)
Réplicas	`"replicas": 30` · línea 10	Casilla “Réplicas”
Duración de cada réplica	`"duracion_h": 1000` · línea 11	Casilla “Duración de cada réplica (h)”
Calentamiento	`"calentamiento_h": 100` · línea 12	Casilla “Calentamiento (h)”
Semilla	`"semilla": 2026` · línea 13	Casilla “Semilla”
Orden de las estaciones	El orden de la lista `"estaciones"` es el recorrido del mueble	—
Escenarios E2 y E3	Iguales a la base; solo cambia Ensamblaje en la línea 7: 12 y 10 min	`--config escenarios/e2.json`
Horas a animar · paso	`--horas` (8 por defecto) y `--paso` (5 min) en `animacion.py`	Casilla “Horas a animar”
Colores de la animación	Constantes en `vista/animacion.py`, p. ej. `TRABAJANDO` (l. 52) y `CUELLO` (l. 56)	—

Comandos, desde 03-simulacion/ con el entorno activo

Instalar (una vez)	`python -m pip install -r requirements.txt`
Panel de la demo	`python panel.py`
Panel con otro escenario	`python panel.py --config escenarios/e2.json`
Simulación base (E1)	`python main.py`
Los tres escenarios	`python main.py --config escenarios/base.json escenarios/e2.json escenarios/e3.json`
Guardar en otra carpeta	`python main.py --config escenarios/e2.json --salida ../04-experimentos`
Solo la animación	`python animacion.py --config escenarios/e2.json --horas 24`
Animación como GIF	`python animacion.py --guardar salidas/graficos/base.gif`
Pruebas	`python -m pytest`

Los resultados quedan en `salidas/` (o en la carpeta de `--salida`), con el nombre del JSON: `resumen-e2.csv`, `linea-e2.csv`, gráficos PNG.

Si preguntan “¿dónde se calcula…?”

CÁLCULO	QUÉ HACE	DÓNDE
Utilización ρ	ρ = λ / μ: lo que llega ÷ capacidad	`modelo/teoria.py` · `mm1` (l. 4)
Cola Lq y espera Wq	$Lq = \rho^2 / (1 - \rho)$ · Wq = Lq / λ	`modelo/teoria.py` · `mm1`
Capacidad de la línea	La menor capacidad entre las estaciones	`teoria.py` · `capacidad_linea` (l. 55)
Cuello de botella	Simulado: la estación con mayor utilización · teórico: la de menor capacidad	`simulacion.py` l. 235 · `teoria.py` l. 60
Ley de Little	Lq = producción × Wq, para comprobar la simulación	`simulacion.py` · `lq_little` (l. 207)
Error porcentual	|simulado − teórico| / teórico × 100	`simulacion.py` · `_error_pct` (l. 214)
La simulación	Llegadas y tiempos al azar (exponenciales); una semilla por réplica	`simulacion.py` · `SimulacionLinea` (l. 79)
Validación de datos	Revisa cada campo del escenario antes de simular	`configuracion.py` · `validar_escenario` (l. 44)

Qué responder si algo sale raro

Escriben letras o un número negativo	El panel muestra “Error: …” y no simula. Todo valor debe ser positivo; réplicas y semilla, enteros.
El calentamiento es mayor que la duración	Se rechaza: el calentamiento debe ser menor que la duración de cada réplica.
Llegan más muebles de los que una estación atiende (ρ ≥ 1)	Simula igual, pero la teoría M/M/1 no existe: sale “—” y aparece un aviso rojo. La cola crece sin límite.
“¿Y con dos operarios?”	El motor lo soporta con `"recursos": 2` en el JSON, pero ya no es M/M/1: la teoría sale “—”. Por eso el panel fija 1 operario.
“¿Por qué los números no son exactos?”	Es una simulación con azar: 30 réplicas promediadas quedan a menos de 5 % de la teoría.
“¿Por qué descartan las primeras horas?”	La línea arranca vacía; el calentamiento evita que ese inicio distorsione los promedios.
La simulación tarda	Para la demo, bajar réplicas o duración en el panel (p. ej. 5 réplicas de 200 h).